

# Project 3: Virtual Memory Library Report

## 1. Implementation details of each VM function

### 1.1 Core design and data structures

The 32 bit library simulates a two level page table on a 4 GB virtual space over a 1 GB physical space.

Global state:

- `physical_memory`: 1 GB mmap region used as simulated RAM
- `page_directory`: lives in physical frame 0 and stores `pde_t` entries
- `physical_bitmap`: 1 bit per physical page, tracks free / used frames
- `virtual_bitmap`: 1 bit per virtual page, tracks allocated regions
- `tlb_store[TLB_ENTRIES]`: direct mapped TLB with {vpn, pfn, valid} entries
- `tlb_lookups`, `tlb_misses`: TLB statistics

Synchronization:

- `vm_mutex` protects page directory, page tables, bitmaps, allocation and free
- `tlb_mutex` protects the TLB array and the TLB statistics

Initialization is guarded with `pthread_once(&vm_init_once, set_physical_mem)`, so even if many threads call `n_malloc` at the same time, the memory system is set up exactly once without races.

The page layout uses a configurable `PGSIZE` with macros for `OFFSET_BITS`, `PTX_BITS`, and `PDX_BITS`, derived per page size in the header. PDE and PTE store a physical frame number shifted by `PFN_SHIFT` plus `PTE_VALID` in the low bit.

### 1.2 `set_physical_mem`

`set_physical_mem` is the one time initialization for the VM:

- Initializes the mutexes
- Computes `total_physical_pages = MEMSIZE / PGSIZE` and `total_virtual_pages = MAX_MEMSIZE / PGSIZE`
- Allocates `physical_memory` with `mmap`
- Allocates zeroed `physical_bitmap`, `virtual_bitmap`, and in the extra credit version also `vpage_meta`
- Places `page_directory` at the beginning of `physical_memory` and zeros that first page
- Marks physical frame 0 used in `physical_bitmap` (page directory page)
- Reserves virtual page 0 in `virtual_bitmap` so `NULL` is never a valid user allocation
- Clears TLB structures and sets `is_initialized = true`

One earlier issue while we were developing and testing that was `set_physical_mem` being called by multiple threads, which could race and double initialize the global state. Switching to `pthread_once` around `set_physical_mem` fixed that so all threads share a consistent VM.

### 1.3 TLB functions: `TLB_add`, `TLB_check`, `print_TLB_missrate`

The TLB follows the instructors' clarifications:

- Each entry caches {vpn, pfn, valid}
- The cache stores physical frame numbers, not `pte_t *`, which matches the guidance that TLBs should not store host pointers
- Indexing is `index = vpn % TLB_ENTRIES`, so this behaves as a direct mapped TLB

`int TLB_add(void *va, void *pa):`

- Computes `vaddr32_t vaddr = VA2U(va)`
- Computes simulated physical offset with a full width pointer difference  
`paddr32_t paddr = (paddr32_t)((uintptr_t)pa - (uintptr_t)physical_memory)`  
This avoids truncation and follows the FAQ about simulated addresses.
- Derives `vpn = vaddr >> OFFSET_BITS` and `pfn = paddr >> OFFSET_BITS`
- Locks `tlb_mutex` and overwrites `tlb_store[index]` with the new {vpn, pfn, valid = true}

`void *TLB_check(void *va):`

- Computes `vpn` and `index` the same way
- Locks `tlb_mutex`, increments `tlb_lookups`, and checks `tlb_store[index]`
- On a hit, reconstructs the physical address as  
`physical_memory + pfn * PGSIZE + offset` and returns that pointer
- On a miss, increments `tlb_misses` and returns `NULL`

`void print_TLB_missrate(void):`

- Prints `tlb_misses / tlb_lookups` as a double

Compulsory misses are handled by the page table walk: the first time a page is accessed, `get_physical_addr` walks the page tables, then calls `TLB_add` before returning. Subsequent accesses hit in the TLB. This matches the idea of compulsory misses from lecture and the Piazza clarifications.

## 1.4 Page table operations: translate and map\_page

`pte_t *translate(pde_t *pgdir, void *va):`

- Computes `pdx = PDX(va)` and `ptx = PTX(va)` using the macros derived from `PGSIZE`
- Looks up `pde = pgdir[pdx]`, checks `PTE_VALID`
- Uses the PFN stored in `pde` to compute the host pointer to the page table:  
`page_table = physical_memory + pt_pfn * PGSIZE`
- Returns `&page_table[ptx]` if that PTE is valid, otherwise returns `NULL`

This function does not interact with the TLB directly. Higher level helpers such as `get_physical_addr` use `TLB_check` first, then call this page table walk on a miss.

`int map_page(pde_t *pgdir, void *va, void *pa):`

- Gets the simulated physical offset with a full width pointer difference from `physical_memory`
- Extracts `pd_index = PDX(va)` and `pt_index = PTX(va)`
- Locks `vm_mutex` while manipulating the page tables

- Allocates a new page table page with `alloc_physical_page` if the pde is empty, zeroes it, and installs its PFN plus `PTE_VALID` into the page directory entry
- Computes `data_pfn` from the physical offset of `pa` and stores a PTE (`data_pfn << PFN_SHIFT`) | `PTE_VALID` in the page table

## 1.5 Allocation and free: `get_next_avail`, `n_malloc`, `n_free`

`void *get_next_avail(int num_pages):`

- Scans the `virtual_bitmap` linearly for `num_pages` consecutive zero bits
- Once found, sets those bits and returns a virtual address `va = index * PGSIZE`
- If no contiguous region exists, returns `NULL`

`void *n_malloc(unsigned int num_bytes):`

- Uses `pthread_once` to ensure the VM is initialized
- Locks `vm_mutex` around the allocation logic

There are two paths:

1. **Small allocations (extra credit internal fragmentation reduction)**
  - If `num_bytes < PGSIZE`, the allocator tries to pack multiple small allocations into a single page.
  - `vpage_meta[vp].kind = 2` marks a page as a “small allocation pool”.
  - `find_small_page_with_space` searches for a small page where `next_free_off + num_bytes <= PGSIZE`.
  - If none exists, `alloc_small_page` reserves a new virtual page, allocates one physical page, maps it, and initializes its metadata with `next_free_off = 0` and `kind = 2`.
  - The allocator returns `vp * PGSIZE + next_free_off`, then bumps `next_free_off` and increments `active_suballocs`.
2. **Large allocations (page based)**
  - For `num_bytes >= PGSIZE`, it computes `num_pages = ceil(num_bytes / PGSIZE)`.
  - Uses `get_next_avail(num_pages)` to reserve contiguous virtual pages.
  - For each page:
    - Allocates a physical page with `alloc_physical_page`.
    - Zeroes the page.
    - Calls `map_page(page_directory, page_va, pa)`.
    - Marks `vpage_meta[vpage].kind = 1` to indicate a full page allocation.
  - On any failure, the code rolls back: clears PTEs, frees physical frames, and clears the corresponding virtual bits, matching the instructors’ answer about cleaning up the virtual bitmap if physical allocation cannot be satisfied.

`void n_free(void *va, int size):`

- Validates parameters and computes `vpage_num = VA2U(va) / PGSIZE`
- Locks `vm_mutex`

There are again two cases:

### 1. Freeing a small allocation inside a suballocation page

- If `size < PGSIZE` and `vpage_meta[vpage_num].kind == 2`, the code treats this as freeing one “slot” in a small allocation page.
- Decrements `active_suballocs`.
- When `active_suballocs` reach zero, the page is truly idle. `free_full_vpage` is called to:
  - Clear the PTE and free the physical frame
  - Clear the virtual bitmap bit
  - Reset the `vpage_meta` entry

### 2. Freeing a larger region (page based)

- Computes `num_pages = ceil(size / PGSIZE)` and aligns the starting VA down to a page boundary.
- For each page covered:
  - Checks the virtual bitmap; if the page was not allocated, prints a warning and skips it.
  - Calls `free_full_vpage` to clear the PTE, free the physical frame, clear the virtual bit, and reset metadata.
- `free_full_vpage` is page based and matches the instructor guidance that partial page frees are promoted to whole pages for this project.

When freeing, the 32 bit version also invalidates any matching TLB entry by scanning `tlb_store` for entries with the same `vpn` and clearing `valid`. That keeps TLB and page tables consistent.

## 1.6 Data movement: `put_data`, `get_data`

Both functions are page aware wrappers on top of the address translation and TLB.

`int put_data(void *va, void *val, int size):`

- Walks from `va` for `size` bytes.
- In each loop iteration:
  - Calls `get_physical_addr` to resolve the current VA to a physical address. `get_physical_addr` probes the TLB first, then walks the page table on a miss, and inserts a TLB entry as a compulsory miss.
  - Computes how many bytes remain in the current page using `PGSIZE - OFF(va)` and caps this at remaining.
  - Copies that chunk with `memcpy`, then advances the virtual address and source pointer.

`void get_data(void *va, void *val, int size)` mirrors this logic for reads. These functions are used directly in `mat_mult` and in the benchmarks.

## 1.7 `mat_mult`

`mat_mult(void *mat1, void *mat2, int size, void *answer):`

- Treats the virtual pointers `mat1`, `mat2`, and `answer` as row major matrices of `uint32_t`.
- For each `i`, `j`, loops over `k`:
  - Builds virtual addresses for `mat1[i][k]` and `mat2[k][j]` using `VA2U(mat)` plus a row major offset.
  - Calls `get_data` twice to fetch `a` and `b`.
  - Accumulates `sum += a * b`.
- Writes the result back to `answer[i][j]` using `put_data`.

This function stresses the VM by touching data across many pages and also drives realistic TLB behavior under concurrent threads in `mtest`.

### 1.8 Extra credit: internal fragmentation reduction (`my_vm3.c`)

The extra credit version introduces `vpage_meta` per virtual page:

- `kind`
  - 0 unused
  - 1 large page based allocation
  - 2 small allocation pool
- `active_suballocs` counts live small allocations in this page
- `next_free_off` is a bump pointer for the next suballocation within the page

Small allocations below one page are packed into shared pages to reduce internal fragmentation. The design is simple on purpose. There is no complex free list, so holes inside a page are not reused, but internal waste across many small allocations is significantly reduced compared to always giving each small allocation its own page.

### 1.9 Extra credit: 64 bit 4 level VM (`my_vm64.c`)

The 64 bit extra credit follows `x86_64` style 4 level page tables over a 48 bit virtual space:

- Virtual addresses use 48 significant bits: [ `L4` | `L3` | `L2` | `L1` | offset ] with 9 bits per level and 12 bit page offset.
- Physical memory is again simulated as a 1 GiB mmap region with 4 KiB pages.
- `pml4` is mapped in physical frame 0.
- `phys_bitmap` tracks 4 KiB frames in the 64 bit physical space.
- `pte64_t` and `pde64_t` store a PFN shifted by `PFN64_SHIFT` plus `PTE64_VALID`.

The TLB for this design:

- `tlb64[TLB64_ENTRIES]` with {`vpn`, `pfn`, `valid`} and a direct mapped index
- `tlb64_lookup` reconstructs host pointers on hits and increments `tlb64_lookups` and `tlb64_misses` appropriately
- `tlb64_add` inserts compulsory misses

Core functions:

- `vm64_init`
  - Allocates 1 GiB physical memory and the bitmap
  - Reserves frame 0 as the PML4 page and zeroes it
  - Clears all `tlb64` entries
  - Marks `vm64_initialized = 1` under `vm64_mutex`
- `map_page_single` and `ensure_pt_level`
  - `ensure_pt_level` allocates a new page table page for each missing level and stores its PFN in the parent entry with `PTE64_VALID`
  - `map_page_single` walks `L4 → L3 → L2 → L1` using `ensure_pt_level` and installs a PTE for the final PFN, then inserts a TLB entry
- `translate_host`
  - First tries the TLB
  - On a miss, walks the four page table levels, checks `PTE64_VALID` at each, reconstructs the host pointer, and calls `tlb64_add`
- `vm64_malloc`
  - Uses a simple bump pointer `next_free_vpage` for the virtual space
  - Computes the number of pages, maps each to a fresh physical frame using `alloc_phys_page` and `map_page_single`, and returns the base virtual address
- `vm64_free`
  - Computes how many pages the region covers and calls `unmap_pages` to clear PTEs and free PFNs. Virtual space is not reused, which simplifies the logic.
- `vm64_put` / `vm64_get`
  - Use `translate_host` inside locked helpers `vm64_put_locked` and `vm64_get_locked` to copy across page boundaries
- `vm64_mat_mult`
  - Works like the 32 bit `mat_mult` but uses `vm64_get_locked` and `vm64_put_locked` on 64 bit virtual addresses

All 64 bit APIs are protected by `vm64_mutex`. `vm64_init` is idempotent and guarded by the same mutex, so the 4 level design is thread safe as required.

## 2. Benchmark output and TLB miss rate

The provided benchmarks for the 32 bit VM are `test` and `mtest`.

Test:

Allocating three arrays of 400 bytes

Virtual memory initialized:

Physical memory: 16384 pages (1024 MB)

Virtual memory: 65536 pages (4096 MB)

Addresses of the allocations (32-bit VAs): 10000, 20000, 30000

Storing integers to generate a `SIZExSIZE` matrix

Fetching matrix elements stored in the arrays

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

Performing matrix multiplication with itself!

Starting matrix multiplication: 5x5 matrices

Matrix multiplication complete

5 5 5 5 5

5 5 5 5 5

5 5 5 5 5

5 5 5 5 5

5 5 5 5 5

Freeing the allocations!

Checking if allocations were freed!

free function works

TLB miss rate 0.007500

Mtest

Prints:Allocated Pointers (as 32-bit VAs):

10000 30000 b0000 20000 c0000 d0000 f0000 80000 90000 a0000 40000 50000 60000 e0000 70000

Initializing some of the memory in multiple threads

Randomly checking a thread allocation

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

1 1 1 1 1

Performing matrix multiplications in multiple threads

Starting matrix multiplication: 5x5 matrices

Starting matrix multiplication: 5x5 matrices

Matrix multiplication complete

Starting matrix multiplication: 5x5 matrices

Starting matrix multiplication: 5x5 matrices

Matrix multiplication complete

Starting matrix multiplication: 5x5 matrices

Matrix multiplication complete

Matrix multiplication complete

Matrix multiplication complete

Randomly checking a thread allocation after matmul

5 5 5 5 5

5 5 5 5 5

5 5 5 5 5

5 5 5 5 5

5 5 5 5 5

Freeing everything in multiple threads

Free Worked! New VA: 10000

TLB miss rate 0.008333

## 5.1 Internal fragmentation reduction

The extra credit for internal fragmentation introduces per virtual page metadata and a small allocation path in `n_malloc` and `n_free`. Small allocations share a page and are tracked via a counter and bump pointer, while large allocations still get whole pages. The MMU and TLB stay unchanged, so the rest of the project continues to see page based mappings and full page frees.

## **5.2 4 level 64 bit page table**

The second extra credit implements a 4 level page table and TLB in `my_vm64.c` with:

- A PML4 root in frame 0
- 4 levels of 9 bit indices and a 12 bit offset
- A direct mapped TLB storing 64 bit vpn and pfn
- Thread safety via a global `vm64_mutex`
- 64 bit equivalents of `malloc`, `free`, `put_data`, `get_data`, and `mat_mul`